

LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 3”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

Padang, 1 April 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	2
2.1 Langkah Kerja.....	2
2.2 StackArray_2511533012	2
2.3 stackArrayDriver_2511533012.....	4
2.4 stackPostfix_2511533012.....	5
2.5 NilaiMaksimum_2511533012.....	6
2.6 SiswaStack_2511533012.....	7
BAB III KESIMPULAN	11
3.1 Ringkasan	11
3.2 Kesimpulan	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan teknologi informasi, pemahaman tentang struktur data dan pemrograman berorientasi objek menjadi hal yang sangat penting bagi seorang programmer. Struktur data digunakan untuk mengelola dan menyimpan data secara efisien, sedangkan konsep objek membantu dalam memodelkan permasalahan ke dalam bentuk program.

Pada praktikum ini, saya mempelajari implementasi kelas dan objek melalui pembuatan program yang merepresentasikan waktu (jam). Program tersebut mencakup berbagai operasi seperti validasi input, konversi ke detik, perbandingan waktu, serta manipulasi data seperti penjumlahan dan pengurangan waktu. Dengan praktikum ini, diharapkan penulis dapat memahami cara kerja objek serta penerapannya dalam pemrograman.

1.2 Tujuan

Tujuan dari praktikum ini adalah untuk memahami konsep dasar struktur data dan pemrograman berorientasi objek, serta mampu mengimplementasikan kelas dan objek dalam bentuk program. Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan dalam membuat operasi-operasi dasar terhadap suatu data secara sistematis dan terstruktur.

1.3 Manfaat

Manfaat dari praktikum ini adalah meningkatkan pemahaman penulis terhadap konsep struktur data dan objek dalam pemrograman. Selain itu, praktikum ini juga membantu dalam melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan suatu permasalahan. Hasil dari praktikum ini diharapkan dapat menjadi dasar dalam pengembangan program yang lebih kompleks di masa mendatang.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan3**.
3. Menambahkan tiga buah file Java, yaitu `stackArray_2511533012.java`, `stackArrayDriver_2511533012.java`, `stackPostfix_2511533012.java`, `NilaiMaksimun_2511533012.java`, `SiswaStack_2511533012.java`,
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 StackArray_2511533012

```
package pekan3_2511533012;

public class stackArray_2511533012 {
    static final int MAX_3012 = 1000;
    int top_3012;
    int a_3012[] = new int [MAX_3012];
    boolean isEmpty_3012() {
        return (top_3012 < 0);
    }
    stackArray_2511533012() {
        top_3012 = -1;
    }
    boolean push_3012(int x_3012) {
        if (top_3012 >= (MAX_3012 - 1)) {
            System.out.println("Stack Overflow");
        }
    }
}
```

```

        return false;
    }
    else {
        a_3012[++top_3012] = x_3012;
        System.out.println(x_3012 + " dimasukkan dalam
stack");
        return true;
    }
}
int pop_3012() {
    if (top_3012 < 0) {
        System.out.println("Stack Underflow");
        return 0;
    } else {
        int x_3012 = a_3012[top_3012--];
        return x_3012;
    }
}
int peek_3012() {
    if (top_3012 < 0) {
        System.out.println("Stack Underflow");
        return 0;
    } else {
        int x_3012 = a_3012[top_3012];
        return x_3012;
    }
}
void print_3012( ) {
    for (int i_3012 = top_3012; i_3012 > -1; i_3012--) {
        System.out.println(" " + a_3012[i_3012]);
    }
}

public static void main(String[] args) {
    // TODO Auto-generated method stub

}
}

```

Analisis:

Program ini merupakan implementasi stack menggunakan array statis. Stack disimpan dalam array dengan kapasitas tertentu, dan menggunakan variabel top untuk menunjukkan posisi elemen teratas. Operasi yang tersedia yaitu push, pop, peek, dan print. Program juga menangani kondisi overflow (stack penuh) dan underflow (stack kosong).

Kesimpulan: Program ini menjelaskan konsep dasar stack secara manual menggunakan array.

2.3 stackArrayDriver_2511533012

```
package pekan3_2511533012;

public class stackArrayDriver_2511533012 {
    public static void main (String[] args) {

        stackArray_2511533012 s = new stackArray_2511533012();
        s.push_3012(10);
        s.push_3012(20);
        s.push_3012(30);
        System.out.println(s.pop_3012() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah : " +
s.peek_3012());
        System.out.println("Elemen pada stack : ");
        s.print_3012();
    }
}
```

Output (2.2. StackArray_2511533012.java) :

```
<terminated> stackArrayDriver_25115330
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack :
    20
    10
```

Analisis :

Program ini berfungsi sebagai driver untuk menguji class stackArray. Program melakukan beberapa operasi seperti menambahkan data ke stack (push), menghapus data (pop), melihat elemen teratas (peek), dan menampilkan

seluruh isi stack. Output yang dihasilkan digunakan untuk memastikan bahwa fungsi stack berjalan dengan benar.

Kesimpulan: Program ini digunakan untuk menguji implementasi stack berbasis array.

2.4 stackPostfix_2511533012

```
package pekan3_2511533012;

import java.util.Stack;
import java.util.Scanner;

public class stackPostfix_2511533012 {
    public static int postfixEvaluate_3012(String expression_3012)
    {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression_3012);
        while (input.hasNext()) {
            if (input.hasNextInt()) {
                s.push(input.nextInt());
            } else {
                String operator_3012 = input.next();
                int operator2_3012 = s.pop();
                int operator1_3012 = s.pop();
                if (operator_3012.equals("+")) {
                    s.push (operator1_3012 +
operator2_3012);
                } else if (operator_3012.equals("-")) {
                    s.push(operator1_3012 -
operator2_3012);
                } else if (operator_3012.equals("*")) {
                    s.push(operator1_3012 *
operator2_3012);
                } else {
                    s.push(operator1_3012 /
operator2_3012);
                }
            }
        }
        input.close();
        return s.pop();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("Hasil postfix = " +
postfixEvaluate_3012("5 2 4 * + 7 -"));
    }
}
```



```
}
```

Output :

```
Hasil postfix = 6
```

Analisis :

Program ini merupakan implementasi **stack untuk operasi postfix (Reverse Polish Notation)**. Meskipun pada kode yang tersedia belum terlihat proses perhitungan postfix secara lengkap, class ini menyediakan dasar struktur stack menggunakan array yang dapat digunakan untuk evaluasi ekspresi postfix. Operasi seperti push, pop, dan peek menjadi komponen utama dalam proses tersebut.

Dalam konsep postfix, setiap operand akan dimasukkan ke dalam stack, kemudian saat operator ditemukan, dua operand teratas akan diambil untuk dilakukan operasi, dan hasilnya dimasukkan kembali ke dalam stack.

2.5 NilaiMaksimum_2511533012

```
package pekan3_2511533012;

import java.util.Stack;

public class NilaiMaksimum_2511533012 {
    public static int max_3012(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue_3012 = s.pop();
```

```

        backup.push(maxValue_3012);
        while (!s.isEmpty()) {
            int next_3012 = s.pop();
            backup.push(next_3012);
            maxValue_3012 = Math.max(maxValue_3012,
next_3012);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue_3012;
    }
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack " + s);
        System.out.println("stack teratas " + s.peek());
        System.out.println("nilai maksimum " + max_3012(s));
    }
}

```

Output :

```

<terminated> NilaiMaksimum_251153
isi stack [70, 12, 20]
stack teratas 20
nilai maksimum 70

```

Analisis :

Program ini bertujuan untuk mencari **nilai maksimum dalam sebuah stack**. Metode max() bekerja dengan cara mengambil seluruh elemen dari stack satu per satu, kemudian membandingkan nilainya untuk mendapatkan nilai terbesar. Selama proses ini, elemen disimpan sementara dalam stack cadangan agar data asli tidak hilang. Setelah selesai, data dikembalikan ke stack awal.

Kesimpulan: Program ini menunjukkan pemanfaatan stack untuk proses pencarian nilai maksimum tanpa menghilangkan isi data.

2.6 SiswaStack_2511533012

```

package pekan3_2511533012;

import java.util.ArrayList;

class Siswa_2511533012 {
    String nama_3012;
    int nim_3012;

    public Siswa_2511533012(String nama_3012, int nim_3012) {
        this.nama_3012 = nama_3012;
        this.nim_3012 = nim_3012;
    }

    @Override
    public String toString() {
        return "Nim : " + nim_3012 + ", Nama : " + nama_3012;
    }
}

public class SiswaStack_2511533012 {
    private ArrayList<Siswa_2511533012> stack;

    public SiswaStack_2511533012() {
        stack = new ArrayList<>();
    }

    public void push_3012(Siswa_2511533012 mhs) {
        stack.add(mhs);
    }

    public Siswa_2511533012 pop_3012() {
        if(!isEmpty_3012()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa_2511533012 peek_3012() {
        if (!isEmpty_3012()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty_3012() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_3012() {
        for (int i_3012 = stack.size() - 1; i_3012 >= 0; i_3012--) {
            System.out.println(stack.get(i_3012));
        }
    }
}

```

```

    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        SiswaStack_2511533012 StudentStack = new
SiswaStack_2511533012();

        Siswa_2511533012 mhs1 = new Siswa_2511533012 ("Ali",
1);
        Siswa_2511533012 mhs2 = new Siswa_2511533012 ("Boby",
2);
        Siswa_2511533012 mhs3 = new Siswa_2511533012
("Charles", 3);

        StudentStack.push_3012(mhs1);
        StudentStack.push_3012(mhs2);
        StudentStack.push_3012(mhs3);

        System.out.println("Siswa di dalam stack : ");
        StudentStack.tampilkanSiswa_3012();

        System.out.println("Siswa Teratas " +
StudentStack.peek_3012());
        System.out.println("Mengeluarkan siswa teratas dari
stack : " + StudentStack.pop_3012());
        System.out.println("Daftar siswa setelah di pop : ");
        StudentStack.tampilkanSiswa_3012();
    }
}

```

Output :

```

5114 @ Javadoc Console X Git Staging
301 <terminated> SiswaStack_2511533012 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe
Siswa di dalam stack :
Nim : 3, Nama : Charles
Nim : 2, Nama : Bobby
Nim : 1, Nama : Ali
Siswa Teratas Nim : 3, Nama : Charles
Mengeluarkan siswa teratas dari stack : Nim : 3, Nama : Charles
Daftar siswa setelah di pop :
Nim : 2, Nama : Bobby
Nim : 1, Nama : Ali

```

Analisis :

Program ini mengimplementasikan struktur data **Stack** menggunakan **ArrayList** untuk menyimpan data siswa. Di dalam program terdapat class Siswa yang memiliki atribut nama dan NIM, serta class SiswaStack yang berfungsi sebagai pengelola stack. Operasi yang digunakan meliputi push untuk

menambahkan data, pop untuk menghapus data teratas, peek untuk melihat data teratas, dan isEmpty untuk mengecek apakah stack kosong.

Program juga menampilkan isi stack dari elemen terakhir ke pertama, sesuai konsep stack (LIFO). **Kesimpulan:** Program ini menunjukkan penerapan stack pada data berbentuk objek.

.

BAB III

KESIMPULAN

3.1 Ringkasan

Pada praktikum pekan ke-3, mahasiswa mempelajari konsep **struktur data Stack (tumpukan)** serta implementasinya menggunakan bahasa pemrograman Java. Stack merupakan struktur data yang bekerja dengan prinsip **LIFO (Last In First Out)**, yaitu data yang terakhir masuk akan menjadi data pertama yang keluar.

Dalam praktikum ini, mahasiswa mengimplementasikan stack menggunakan dua pendekatan, yaitu menggunakan **array** dan **ArrayList**. Selain itu, dibuat juga program yang mengelola data objek seperti siswa dalam bentuk stack, serta program driver untuk menguji fungsi-fungsi seperti *push*, *pop*, *peek*, dan *print*.

Melalui berbagai program tersebut, mahasiswa dapat memahami bagaimana cara menyimpan, mengakses, dan menghapus data dalam struktur stack, serta memahami penerapannya dalam kasus sederhana.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data **Stack** sangat berguna dalam pengelolaan data yang membutuhkan konsep urutan tertentu, khususnya dengan prinsip LIFO. Implementasi stack menggunakan array maupun ArrayList memiliki fungsi yang sama, namun berbeda dalam cara pengelolaan memorinya.

Dengan praktikum ini, mahasiswa diharapkan mampu memahami konsep dasar stack serta dapat mengimplementasikannya dalam program Java, baik menggunakan tipe data sederhana maupun objek, sebagai dasar untuk mempelajari struktur data yang lebih kompleks.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials," 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>.
- [2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Website_2511533012.java

```
package pekan3_2511533012;

public class Website_2511533012 {
    private String judul_3012;
    private String url_3012;

    // Constructor
    public Website_2511533012(String judul, String url) {
        this.judul_3012 = judul;
        this.url_3012 = url;
    }

    // Getter
    public String getJudul() {
        return judul_3012;
    }

    public String getUrl() {
        return url_3012;
    }

    // Setter
    public void setJudul(String judul) {
        this.judul_3012 = judul;
    }

    public void setUrl(String url) {
        this.url_3012 = url;
    }
}
```

2. Browser_2511533012.java

```
package pekan3_2511533012;

import java.util.Stack;
import java.util.Scanner;

public class Browser_2511533012 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Stack<Website_2511533012> history = new Stack<>();

        int pilihan;

        do {
```



```

System.out.println("\n=== Browser History NIM: 2511533012
===");

System.out.println("1. Kunjungi Website (Push)");
System.out.println("2. Tombol Back (Pop)");
System.out.println("3. Lihat Halaman Aktif (Peek)");
System.out.println("4. Cek Status History");
System.out.println("5. Keluar");
System.out.print("Pilihan: ");
pilihan = input.nextInt();
input.nextLine();

switch (pilihan) {
    case 1:
        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();
        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        history.push(new Website_2511533012(judul, url));
        System.out.println("Berhasil mengunjungi
halaman!");
        break;

    case 2:
        if (!history.isEmpty()) {
            Website_2511533012 w = history.pop();
            System.out.println("Kembali dari: " +
w.getJudul());
        } else {
            System.out.println("History kosong!");
        }
        break;

    case 3:
        if (!history.isEmpty()) {
            Website_2511533012 w = history.peek();
            System.out.println("Halaman aktif: " +
w.getJudul() +
" (" + w.getUrl() + ")");
        } else {
            System.out.println("Tidak ada halaman!");
        }
        break;

    case 4:
        System.out.println("Jumlah history: " +
history.size());
        System.out.println("Kosong? " + history.isEmpty());
        break;
}

} while (pilihan != 5);
}

```

```
}
```

3. Output dan Penjelasan Syntax

```
=== Browser History NIM: 2511533012 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: netflix.com
Masukkan URL: http://netflix.com/
Berhasil mengunjungi halaman!

3 === Browser History NIM: 2511533012 ===
3 1. Kunjungi Website (Push)
5 2. Tombol Back (Pop)
3 3. Lihat Halaman Aktif (Peek)
1 4. Cek Status History
1 5. Keluar
Pilihan: 2
Kembali dari: netflix.com

=== Browser History NIM: 2511533012 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Tidak ada halaman!
```

```
25 === Browser History NIM: 2511533012 ===
53 1. Kunjungi Website (Push)
01 2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 0
Kosong? true

=== Browser History NIM: 2511533012 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
```

Penjelasan:

Pada pekan ini, program menggunakan struktur data **Stack** yang menerapkan konsep LIFO (Last In, First Out), yaitu data yang terakhir masuk akan menjadi yang pertama keluar. Stack diimplementasikan menggunakan `java.util.Stack` untuk mensimulasikan riwayat browser. Program memiliki kelas ADT Website yang berisi atribut judul dan URL, lengkap dengan constructor,

getter, dan setter. Pada kelas driver, operasi utama Stack digunakan seperti `push()` untuk menambahkan halaman baru ke tumpukan saat pengguna mengunjungi website, `pop()` untuk menghapus halaman terakhir saat tombol back ditekan, dan `peek()` untuk melihat halaman yang sedang aktif tanpa menghapusnya. Selain itu, method `isEmpty()` digunakan untuk mengecek apakah stack kosong agar program tidak error saat melakukan pop, serta `size()` untuk melihat jumlah riwayat yang tersimpan. Dari praktikum ini, konsep penting yang dipahami adalah bagaimana Stack bekerja dalam kehidupan nyata, khususnya pada sistem navigasi browser.